

Reviewer Pack — Fourier Coefficient Sum Inverse Proportionality to DPLL Tree...

Entry 2adc13aba337 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 10:43:31 UTC

Fourier Coefficient Sum Inverse Proportionality to DPLL Tree Size

Entry ID: 2adc13aba337 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 10:43:31 UTC

1. Conjecture

Field A (mathematical branch): Harmonic Analysis on Finite Groups **Field B** (complexity object): DPLL Search Tree Size

Statement:

For a random CNF formula Φ with n variables and m clauses, the expected size of the DPLL search tree $T(\Phi)$ satisfies $E[T(\Phi)] = \Theta(1 / \sum_{S} |f_S|)$, where f_S is the Fourier coefficient of the boolean function f corresponding to Φ at subset S .

Rationale (proposer's reasoning):

The Fourier coefficients capture the influence of variable subsets on the function's output. A higher sum of absolute coefficients indicates more balanced influence, potentially leading to a smaller DPLL tree as the algorithm can prune branches more effectively.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d6cd337734be2dc8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def generate_cnf(n, m):
    cnf = []
    for _ in range(m):
        clause = set()
        while len(clause) < 3:
            var = random.randint(1, n)
            sign = random.choice([-1, 1])
            if (var, sign) not in clause and (-var, -sign) not in clause:
                clause.add((var, sign))
        cnf.append(clause)
    return cnf

def dpll(cnf, assignment):
    if not cnf:
        return True
    unit_clauses = [c for c in cnf if len(c) == 1]
    if unit_clauses:
        var, sign = unit_clauses[0][0], unit_clauses[0][1]
        if (var, sign) in assignment or (-var, -sign) in assignment:
            return False
        assignment[var] = sign
        cnf = [c for c in cnf if var not in c and -var not in c]
    pure_literals = {}
    for literal in set(var for clause in cnf for var, _ in clause):
        pos_count = sum(sign == 1 for _, sign in clause if literal == abs(literal))
        neg_count = sum(sign == -1 for _, sign in clause if literal == abs(literal))
        if pos_count == 0:
            pure_literals[literal] = -1
        elif neg_count == 0:
            pure_literals[literal] = 1
    if pure_literals:
        var, sign = next((k, v) for k, v in pure_literals.items())
        if (var, sign) in assignment or (-var, -sign) in assignment:
            return False
        assignment[var] = sign
        cnf = [c for c in cnf if var not in c and -var not in c]
```

```

for literal in range(1, n + 1):
    if literal not in assignment and -literal not in assignment:
        if dpll(cnf, {**assignment, literal: 1}):
            return True
        if dpll(cnf, {**assignment, literal: -1}):
            return True
return False

def fourier_coefficient(cnf, subset):
    n = len(cnf)
    assignment = {var: (i >> j) & 1 for j in range(n) for var in cnf[j] if var[0] == j + 1}
    return (-1) ** sum(assignment[var] != sign for var, sign in subset)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = random.randint(n * 2, n * 3)
    cnf = generate_cnf(n, m)

    dpll_size = sum(dpll(cnf, {}) for _ in range(10))
    subsets = list(combinations(range(1, n + 1), 1)) + list(combinations(range(1, n + 1), 2))
    fourier_sum = sum(fourier_coefficient(cnf, subset) for subset in subsets)

    metric_value = dpll_size / abs(fourier_sum)
    instances_tested = 10
    conjecture_holds = True if metric_value > 0 else False
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "DPLL Tree Size",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])

```

```
print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9bbbe86d.py", line 95, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9bbbe86d.py", line 72, in run_trial
    dpll_size = sum(dpll(cnf, { }) for _ in range(10))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9bbbe86d.py", line 72, in <genexpr>
    dpll_size = sum(dpll(cnf, { }) for _ in range(10))
                  ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9bbbe86d.py", line 41, in dpll
    pos_count = sum(sign == 1 for _, sign in clause if literal == abs(literal))
                  ^^^^^^
```

NameError: name 'clause' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to undefined variable 'clause' in dpll function, preventing data collection
| next: Fix the dpll function's clause reference and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109691
2	propose	ollama_remote	qwen3:8b	0	0	102151
3	preregistration	ollama_remote	qwen3:8b	0	0	23844
4	novelty	ollama_remote	qwen3:8b	0	0	22281
5	novelty	ollama_remote	qwen3:8b	0	0	16197
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17969
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12560
8	judge	ollama_remote	qwen3:8b	0	0	13889

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 318583 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/2adc13aba337.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/2adc13aba337.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/2adc13aba337.tar.gz (if generated)